

WEEK-1

Session-1: Introduction to functional programming concepts

Java Lambda Expressions

```
import java.io.*;

import java.util.*;

interface PerformOperation {

    boolean check(int a);

}

class MyMath {

    public static boolean checker(PerformOperation p, int num) {

        return p.check(num);

    }

    // Write your code here

    public PerformOperation isOdd() {

        return (int a) -> a % 2 != 0;

    }

    public PerformOperation isPrime() {

        return (int a) -> {

            if (a < 2) {

                return false;

            }

            for (int i = 2; i * i <= a; i++) {

                if (a % i == 0) {

                    return false;

                }

            }

            return true;

        }

    }

}
```

```
};  
}
```

```
public PerformOperation isPalindrome() {  
    return (int a) -> {  
        int original = a;  
        int reverse = 0;  
  
        while (a > 0) {  
            int digit = a % 10;  
            reverse = reverse * 10 + digit;  
            a /= 10;  
        }  
  
        return original == reverse;  
    };  
}
```

```
public class Solution {  
  
    public static void main(String[] args) throws IOException {  
        MyMath ob = new MyMath();  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
        int T = Integer.parseInt(br.readLine());  
        PerformOperation op;  
        boolean ret = false;  
        String ans = null;  
        while (T--> 0) {  
            String s = br.readLine().trim();  
            StringTokenizer st = new StringTokenizer(s);
```

```
int ch = Integer.parseInt(st.nextToken());  
int num = Integer.parseInt(st.nextToken());  
if (ch == 1) {  
    op = ob.isOdd();  
    ret = ob.checker(op, num);  
    ans = (ret) ? "ODD" : "EVEN";  
} else if (ch == 2) {  
    op = ob.isPrime();  
    ret = ob.checker(op, num);  
    ans = (ret) ? "PRIME" : "COMPOSITE";  
} else if (ch == 3) {  
    op = ob.isPalindrome();  
    ret = ob.checker(op, num);  
    ans = (ret) ? "PALINDROME" : "NOT PALINDROME";  
  
}  
System.out.println(ans);  
}  
}  
}
```

Congratulations

You solved this challenge. Would you like to challenge your friends?

[Next Challenge](#)

✓ Test case 0

✓ Test case 1

✓ Test case 2

Compiler Message

Success

Input (stdin)

```
1 5
2 1 4
3 2 5
4 3 898
5 1 3
6 2 12
```

[Download](#)

Expected Output

```
- 1 2 3 4 5 6
```

[Download](#)

Java Comparator

```
import java.util.*;
```

```
// Write your Checker class here
```

```
class Checker implements Comparator<Player> {
```

```
    @Override
```

```
    public int compare(Player a, Player b) {
```

```
        // Higher score comes first
```

```
        if (a.score != b.score) {
```

```
            return b.score - a.score;
```

```
        }
```

```
        // Same score: alphabetical order by name
```

```
        return a.name.compareTo(b.name);
```

```
    }
```

```
}
```

```
class Player{  
    String name;  
    int score;  
  
    Player(String name, int score){  
        this.name = name;  
        this.score = score;  
    }  
}
```

```
class Solution {  
  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        int n = scan.nextInt();  
  
        Player[] player = new Player[n];  
        Checker checker = new Checker();  
  
        for(int i = 0; i < n; i++){  
            player[i] = new Player(scan.next(), scan.nextInt());  
        }  
        scan.close();  
        Arrays.sort(player, checker);  
        for(int i = 0; i < player.length; i++){  
            System.out.printf("%s %s\n", player[i].name, player[i].score);  
        }  
    }  
}
```

Submitted Code

Language: Java 8

[Open in editor](#)

✓ Test case 0

Compiler Message

Success

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

[Download](#)

Expected Output

```
1 aleksa 150
2 amy 100
```

[Download](#)

Session-2: Lambda Expressions

Java Sort

```
import java.util.*;

class Student{
    private int id;
    private String fname;
    private double cgpa;
    public Student(int id, String fname, double cgpa) {
        super();
        this.id = id;
        this.fname = fname;
        this.cgpa = cgpa;
    }
    public int getId() {
        return id;
    }
}
```

```

    public String getFname() {
        return fname;
    }
    public double getCgpa() {
        return cgpa;
    }
}

```

//Complete the code

```

public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());

        List<Student> studentList = new ArrayList<Student>();
        while(testCases>0){
            int id = in.nextInt();
            String fname = in.next();
            double cgpa = in.nextDouble();

            Student st = new Student(id, fname, cgpa);
            studentList.add(st);

            testCases--;
        }
        Collections.sort(studentList, new Comparator<Student>() {
            @Override
            public int compare(Student a, Student b) {
                if (a.getCgpa() != b.getCgpa()) {
                    return Double.compare(b.getCgpa(), a.getCgpa());
                }
            }
        });
    }
}

```

```

    }

    int nameCompare = a.getFname().compareTo(b.getFname());

    if (nameCompare != 0) {
        return nameCompare;
    }

    return Integer.compare(a.getId(), b.getId());
}

});

for(Student st: studentList){
    System.out.println(st.getFname());
}
}
}

```

Submitted Code

Language: Java 8

[Open in editor](#)

Test case 0

Compiler Message

Success

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Input (stdin)

```

1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76

```

[Download](#)

Expected Output

```

1 Ashis
2
3
4
5
6

```

[Download](#)

Session-3: Anonymous Functions & Functional Interfaces

Sorting: Comparator

```
import java.util.*;
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Checker implements Comparator<Player> {
```

```
    // complete this method
```

```
    public int compare(Player a, Player b) {
```

```
        return Comparator.comparing((Player p) -> p.score, Comparator.reverseOrder())
```

```
            .thenComparing(p -> p.name)
```

```
            .compare(a, b);
```

```
    }
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner scan = new Scanner(System.in);
```

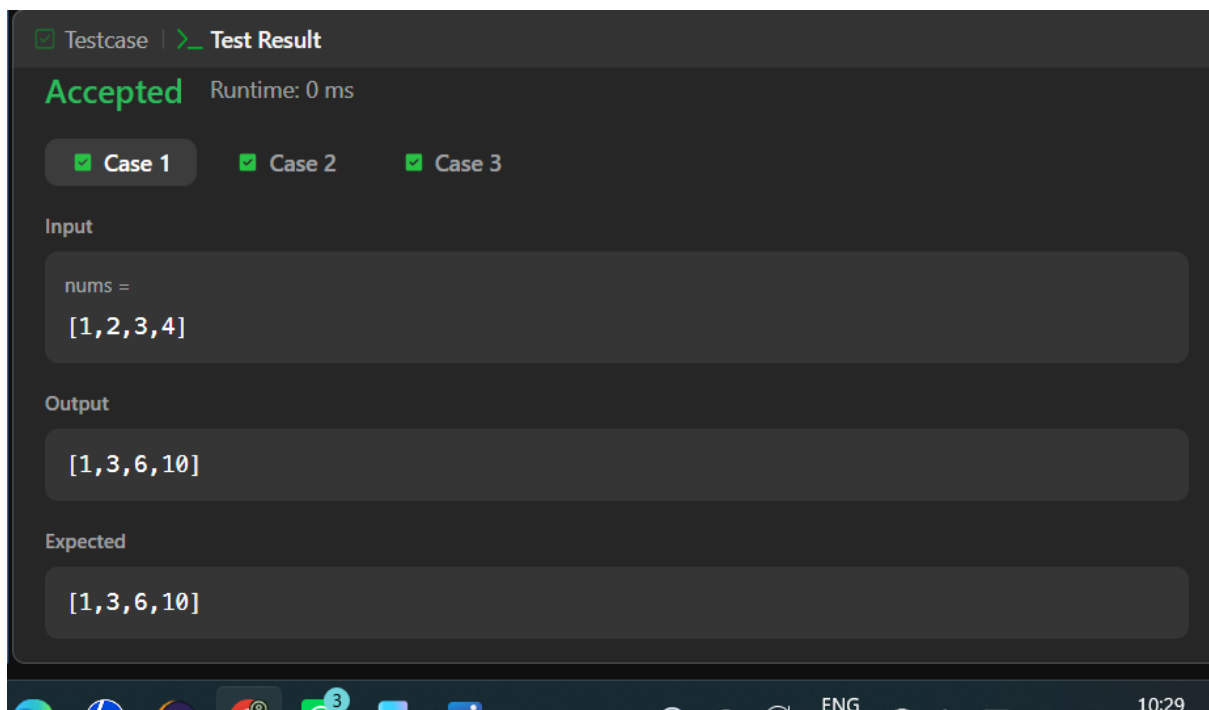
```
        int n = scan.nextInt();
```

```
        Player[] player = new Player[n];
```


Session-4: Higher-Order Functions

Running Sum of 1D Array

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] = nums[i] + nums[i - 1];  
        }  
        return nums;  
    }  
}
```



Richest Customer Wealth

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int maxWealth = 0;  
  
        for (int i = 0; i < accounts.length; i++) {
```

```

        int wealth = 0;

        for (int j = 0; j < accounts[i].length; j++) {
            wealth += accounts[i][j];
        }

        maxWealth = Math.max(maxWealth, wealth);
    }

    return maxWealth;
}
}

```

☒ Testcase
 >_ Test Result

Accepted
Runtime: 0 ms

☒ Case 1
 ☒ Case 2
 ☒ Case 3

Input

accounts =
 [[1,2,3],[3,2,1]]

Output

6

Expected

6

Session-5: Higher-Order Functions

Squares of a Sorted Array

```

class Solution {
    public int[] sortedSquares(int[] nums) {

```

```

int n = nums.length;

int[] result = new int[n];

int left = 0;

int right = n - 1;

int index = n - 1;

while (left <= right) {

    if (Math.abs(nums[left]) > Math.abs(nums[right])) {

        result[index] = nums[left] * nums[left];

        left++;

    } else {

        result[index] = nums[right] * nums[right];

        right--;

    }

    index--;

}

return result;

}

```

☒ Testcase
 [Test Result](#)

Accepted
Runtime: 0 ms

☒ Case 1
 ☒ Case 2

Input

```

nums =
[-4,-1,0,3,10]
        
```

Output

```

[0,1,9,16,100]
        
```

Expected

```

[0,1,9,16,100]
        
```

Find Pivot Index

```
class Solution {  
    public int pivotIndex(int[] nums) {  
        int totalSum = 0;  
  
        // Calculate total sum  
        for (int num : nums) {  
            totalSum += num;  
        }  
  
        int leftSum = 0;  
  
        // Find pivot index  
        for (int i = 0; i < nums.length; i++) {  
            int rightSum = totalSum - leftSum - nums[i];  
  
            if (leftSum == rightSum) {  
                return i;  
            }  
  
            leftSum += nums[i];  
        }  
  
        return -1;  
    }  
}
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,7,3,6,5,6]
```

Output

```
3
```

Expected

```
3
```